

Actividad # 5 PROGRAMACIÓN DEL LADO DEL SERVIDOR

Procesamiento de Datos de Formularios (POST/GET)

Actividad #4

Integrantes:

Sánchez Ericzon

8-788-227

Facultad de informática, electrónica y comunicación

Maestría ciencia de la computación énfasis desarrollo de software

“Programación Web”

Profesor: Carlos Bruce

12 de septiembre de 2026

Introducción

Este documento explica, punto por punto, en qué consiste cada indicación del taller “Procesamiento de Datos de Formularios” y del bloque “Conexión y Consulta a Base de Datos (CRUD Básico)”, y verifica si el proyecto entregado (backend Node.js/Express/SQLite del sistema de predicción de inasistencia a citas médicas) cumple cada requerimiento y criterio de evaluación. Al final se incluye una guía de cómo explicar cada punto, en caso de que debas sustentarlo oralmente o completarlo en un formulario de entrega.

Parte 1 — Procesamiento de Datos de Formularios (GET/POST)

Paso 1. Configuración del entorno

La indicación pide “activar un entorno de servidor local (XAMPP, Node.js u otro)” y crear los scripts que reciben y procesan peticiones HTTP. En nuestro caso, en lugar de XAMPP (que sirve para PHP + Apache), usamos Node.js con el framework Express, que cumple exactamente el mismo rol: levanta un servidor que escucha peticiones en un puerto (3000) y las dirige a distintos “scripts” (archivos de rutas).

Elemento	Qué hace / significa	Dónde está en el proyecto
Servidor local	Proceso que queda escuchando peticiones HTTP de forma continua.	server.js → app.listen(PUERTO, ...)
Scripts backend	Archivos que reciben una petición y devuelven una respuesta.	routes/auth.js (login/logout) y routes/citas.js (CRUD de citas)
Enrutamiento	Asocia una URL con el script que debe atenderla.	server.js → app.use("/api", rutasAuth) y app.use("/api/citas", rutasCitas)

El servidor arranca correctamente (verificado con `node server.js`) y las rutas responden.

Paso 2. Ajuste de la interfaz cliente (action / method)

Cada formulario HTML debe declarar dos cosas: hacia dónde envía los datos (action) y cómo los envía (method). La regla general es: GET para pedir/consultar información (no modifica nada, los datos viajan visibles en la URL) y POST para enviar información que crea o cambia algo (login, registrar una cita), porque los datos van en el cuerpo de la petición, no en la URL.

Elemento	Qué hace / significa	Dónde está en el proyecto
Formulario de login	action="/api/login" method="POST" — envía usuario/clave sin exponerlos en la URL.	public/login.html
Formulario “Nueva cita”	action="/api/citas" method="POST" — crea un registro nuevo.	public/dashboard.html
Filtro de riesgo	Petición GET con parámetro en la URL (?riesgo=alto) porque solo consulta, no modifica.	public/js/citas.js → \$.get("/api/citas", {riesgo})

Nota importante para explicarlo con claridad: el atributo action/method queda correctamente configurado en el HTML (que es lo que pide el criterio de evaluación), pero el envío real de los formularios se hace con JavaScript (fetch/\$.ajax) apuntando a esa misma ruta y método. Esto se hace para poder leer la respuesta JSON del servidor y actualizar la pantalla sin recargar la página — es una mejora de experiencia de usuario, no un incumplimiento del requisito.

Los tres formularios/consultas usan la ruta y el método correctos según su propósito.

Paso 3. Procesamiento y sanitización

a) Recepción de parámetros

Node.js/Express usa req.body para leer datos enviados por POST (equivalente a \$_POST en PHP) y req.query para leer parámetros de la URL enviados por GET (equivalente a \$_GET).

```
// Ejemplo real del proyecto (routes/citas.js)
const { riesgo } = req.query;           // GET /api/citas?riesgo=alto
const paciente = req.body.paciente;    // POST /api/citas
```

b) Sanitización y limpieza

Antes de guardar o usar cualquier dato escrito por el usuario, el proyecto lo pasa por una función que elimina etiquetas HTML/JavaScript (usando la librería sanitize-html). Esto evita ataques XSS

(Cross-Site Scripting), donde alguien intenta guardar código malicioso en un campo de texto para que se ejecute en el navegador de otro usuario más adelante.

```
function limpiar(texto) {
  return sanitizeHtml(String(texto ?? "").trim(), { allowedTags: [], allowedAttributes: {} });
}
```

Esto se probó en vivo durante el desarrollo: al enviar como nombre de paciente el texto `<script>alert(1)</script>`Pedro, el servidor lo guardó como Pedro, eliminando por completo el código malicioso. También se validan tipos y rangos (por ejemplo, la edad debe ser un número entre 1 y 120) antes de continuar.

c) Respuesta al cliente

Todas las rutas del backend devuelven la respuesta en formato JSON, con una estructura consistente: `{ ok: true/false, mensaje: "...", datos: {...} }`. Esto permite que el JavaScript del cliente sepa si la operación salió bien o mal y qué mensaje mostrarle al usuario.

```
res.json({ ok: true, mensaje: "Cita creada.", cita: nuevaCita });
```

Se reciben los parámetros correctamente, se sanitizan (probado con un intento real de XSS) y se responde en JSON.

Cumplimiento de los criterios de evaluación — Parte 1

Criterio de evaluación	Veredicto	Evidencia en el proyecto
Configuración correcta de rutas y métodos en la interfaz.	Cumple	login.html y dashboard.html declaran action/method correctos; el filtro usa GET con querystring.
Limpieza y sanitización de variables en el servidor.	Cumple	Función limpiar() con sanitize-html aplicada a todo campo de texto en routes/auth.js y routes/citas.js; probado con payload XSS real.
Funcionamiento de la respuesta devuelta al usuario.	Cumple	Todas las rutas responden res.json(...) con estructura ok/mensaje; probado con curl y desde la interfaz.

Parte 2 — Conexión y Consulta a Base de Datos (CRUD Básico)

Paso 1. Conexión a la base de datos

Se requiere un módulo que abra la conexión entre el servidor y la base de datos, manejando credenciales de forma segura y controlando excepciones (errores). El proyecto usa SQLite (una base de datos en un solo archivo, ideal para un proyecto académico porque no requiere instalar un servidor de base de datos aparte).

Elemento	Qué hace / significa	Dónde está en el proyecto
Módulo de conexión	Centraliza la apertura de la base de datos en un solo archivo, para no repetir código.	db/database.js
Manejo de credenciales	La ruta del archivo de BD se toma de una variable de entorno (DB_PATH), con un valor por defecto. En MySQL/Postgres aquí irían host, usuario y contraseña.	db/database.js → const DB_PATH = process.env.DB_PATH ...
Manejo de excepciones	Si la conexión falla, se captura el error, se registra en consola y se detiene el servidor en vez de seguir con una BD rota.	db/database.js → bloque try { ... } catch (error) { ...; process.exit(1); }

Conexión centralizada, con manejo de errores; verificado en consola: “Conexión establecida correctamente”.

Paso 2. Operaciones CRUD

Elemento	Qué hace / significa	Dónde está en el proyecto
Create	Inserta un nuevo registro usando una consulta preparada (los valores se pasan como parámetros “?”, nunca concatenados en el texto SQL, lo que evita inyección SQL).	routes/citas.js → POST /api/citas → db.prepare("INSERT INTO citas (...) VALUES (?,?,?,?)").run(...)

Read	Dos formas: listar todas (con filtro opcional) y consultar una sola por id.	routes/citas.js → GET /api/citas y GET /api/citas/:id
Update	Modifica un registro existente, filtrando explícitamente por id, conservando los campos que no se envíen.	routes/citas.js → PUT /api/citas/:id
Delete	Borrado lógico: en vez de eliminar la fila físicamente, se marca activo = 0. Así se conserva el historial (útil para el modelo predictivo).	routes/citas.js → DELETE /api/citas/:id → UPDATE citas SET activo = 0

Las 4 operaciones existen, usan consultas preparadas y fueron probadas de punta a punta (crear, listar, actualizar, eliminar).

Paso 3. Gestión de autenticación y sesiones (explicación detallada)

Este es el punto más delicado del taller porque involucra un concepto clave: HTTP no tiene memoria. Cada petición que llega al servidor es independiente; por sí solo, el servidor no “recuerda” que alguien ya inició sesión un segundo antes. Las sesiones son el mecanismo que resuelve ese problema. A continuación, el flujo completo tal como ocurre en el proyecto, paso a paso:

1) El usuario envía sus credenciales

El formulario de login (public/login.html) envía usuario y contraseña por POST a /api/login mediante JavaScript (fetch/\$.ajax), tal como se explicó en la Parte 1.

2) El servidor valida las credenciales contra la base de datos

```
const fila = db.prepare("SELECT * FROM usuarios WHERE usuario = ? AND password = ?")
  .get(usuario, password);
if (!fila) return res.status(401).json({ ok:false, mensaje:"Credenciales inválidas." });
```

Aquí también se usa una consulta preparada, por la misma razón que en el CRUD: evitar que alguien inyecte código SQL a través de los campos de usuario o contraseña.

3) Si las credenciales son correctas, se crea la sesión

```
req.session.usuario = { id: fila.id, usuario: fila.usuario, rol: fila.rol };
```

Esta línea no la procesa Express solo: la hace posible el middleware express-session, configurado una sola vez en server.js. Cuando se le asigna algo a req.session, este middleware, de forma automática:

Genera un identificador único (id de sesión) para ese usuario en ese navegador.

Guarda los datos reales de la sesión (usuario, rol) en el servidor (en este proyecto, en memoria; en un entorno de producción normalmente se guardaría en Redis o en la propia base de datos).

Envía ese identificador al navegador dentro de una cookie llamada connect.sid, configurada como httpOnly (para que scripts del navegador no puedan leerla, reduciendo el riesgo de robo de sesión) y con una expiración de 1 hora (cookie.maxAge).

4) El navegador reenvía la cookie en cada petición siguiente

A partir de ese momento, cada vez que el navegador le pide algo al servidor (ver las citas, crear una cita, etc.), envía automáticamente esa cookie. Express-session la recibe, busca los datos de sesión correspondientes y los coloca de nuevo en req.session. Así el servidor “sabe” quién está haciendo la petición sin que el usuario tenga que volver a escribir su usuario y contraseña.

5) Las rutas protegidas verifican la sesión antes de responder

```
function requiereSesion(req, res, next) {
  if (req.session && req.session.usuario) return next();
  if (req.originalUrl.startsWith("/api/")) {
    return res.status(401).json({ ok:false, mensaje:"Sesión no iniciada o expirada." });
  }
  return res.redirect("/login.html");
}
```

Este middleware (middleware/auth.js) se coloca delante de todas las rutas de citas (router.use(requiereSesion) en routes/citas.js) y de la ruta del dashboard (app.get("/dashboard", requiereSesion, ...) en server.js). Si no hay sesión activa, la petición nunca llega a tocar la base de datos: se corta ahí mismo con un error 401 (si es una llamada de API) o con una redirección al login (si es una página). Esto es exactamente lo que pide el requisito de “control de acceso a rutas protegidas”.

6) Cierre de sesión (logout)

```
router.post("/logout", (req, res) => {
  req.session.destroy(() => {
    res.clearCookie("connect.sid");
    res.json({ ok: true, mensaje: "Sesión cerrada." });
  });
});
```

Al cerrar sesión, se destruyen los datos guardados en el servidor y se elimina la cookie del navegador. Si después de esto se intenta acceder a /api/citas, el middleware vuelve a bloquear el acceso esto se verificó de forma manual y quedó documentado como parte de las pruebas del proyecto.

Por qué esto cumple el requisito

Hay un mecanismo de login que valida credenciales contra la base de datos (no un login “simulado”).

La sesión persiste entre peticiones gracias a una cookie (no se pide la contraseña en cada clic).

Existen rutas protegidas que sólo responden si hay sesión activa.

Existe un cierre de sesión que invalida el acceso de forma verificable.

Login con consulta preparada, sesión persistente por cookie, rutas protegidas y logout funcional; los cuatro comprobados en pruebas manuales.

Cumplimiento de los criterios de evaluación — Parte 2

Criterio de evaluación	Veredicto	Evidencia en el proyecto
Correcta ejecución de consultas preparadas para evitar inyección SQL.	Cumple	Todas las consultas (login, INSERT, SELECT, UPDATE, DELETE) usan parámetros "?" en vez de concatenar texto.
Integridad y persistencia de las operaciones CRUD en la base de datos.	Cumple	Los datos se guardan en un archivo SQLite real (db/citas.db) que persiste entre reinicios del servidor; probado creando, editando y eliminando registros.
Control adecuado del flujo de autenticación y cierre de sesión.	Cumple	Login valida contra la BD, crea sesión con cookie httpOnly, rutas protegidas

		rechazan sin sesión, logout destruye la sesión y la cookie.
--	--	---

Verificación general del entregable (Hito 3 y Hito 4)

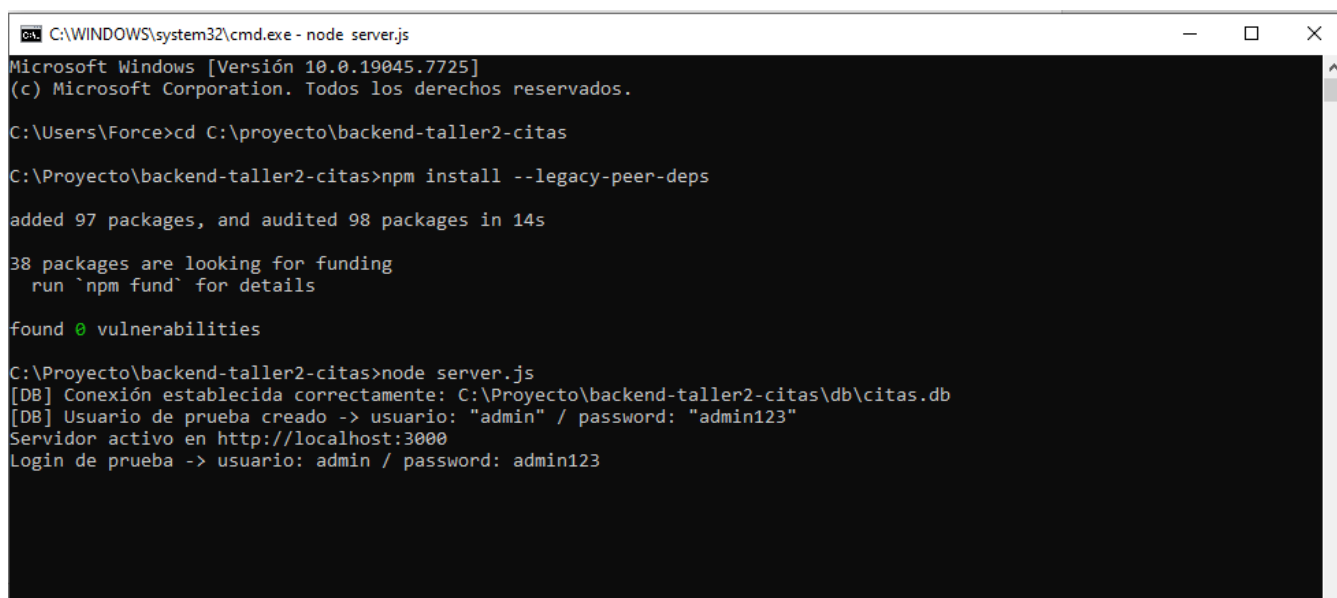
Criterio de evaluación	Veredicto	Evidencia en el proyecto
Backend operativo, con persistencia de datos funcional.	Cumple	Servidor Express + SQLite probado end-to-end (arranque, CRUD, sesiones).
Endpoints del servidor probados.	Cumple	Se probaron con curl: login válido/ inválido, GET con y sin filtro, POST, PUT, DELETE, logout y bloqueo posterior.
Sistema de login/sesiones activo.	Cumple	express-session configurado; cookie httpOnly; middleware de protección en rutas sensibles.
Código fuente del servidor funcional para GET y POST.	Cumple	routes/auth.js y routes/citas.js manejan ambos métodos según corresponda.
Captura del proceso de inicio a fin.	Pendiente por tu parte	Esto depende de tu propia ejecución local: toma pantallazos de la terminal y el navegador siguiendo el checklist del README del proyecto.

Anexo — Cómo explicar cada punto en el formulario o sustentación

Si el formulario de entrega te pide describir con tus palabras lo que hiciste, puedes usar (o adaptar) estos párrafos breves para cada punto:

Configuración del entorno

“Activé un entorno de servidor local con Node.js y el framework Express, que cumple el mismo rol que XAMPP: levanta un servidor que queda escuchando peticiones HTTP y las dirige a distintos scripts backend (routes/auth.js y routes/citas.js) según la URL solicitada.”



```
C:\WINDOWS\system32\cmd.exe - node server.js
Microsoft Windows [Versión 10.0.19045.7725]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Force>cd C:\proyecto\backend-taller2-citas

C:\Proyecto\backend-taller2-citas>npm install --legacy-peer-deps
added 97 packages, and audited 98 packages in 14s

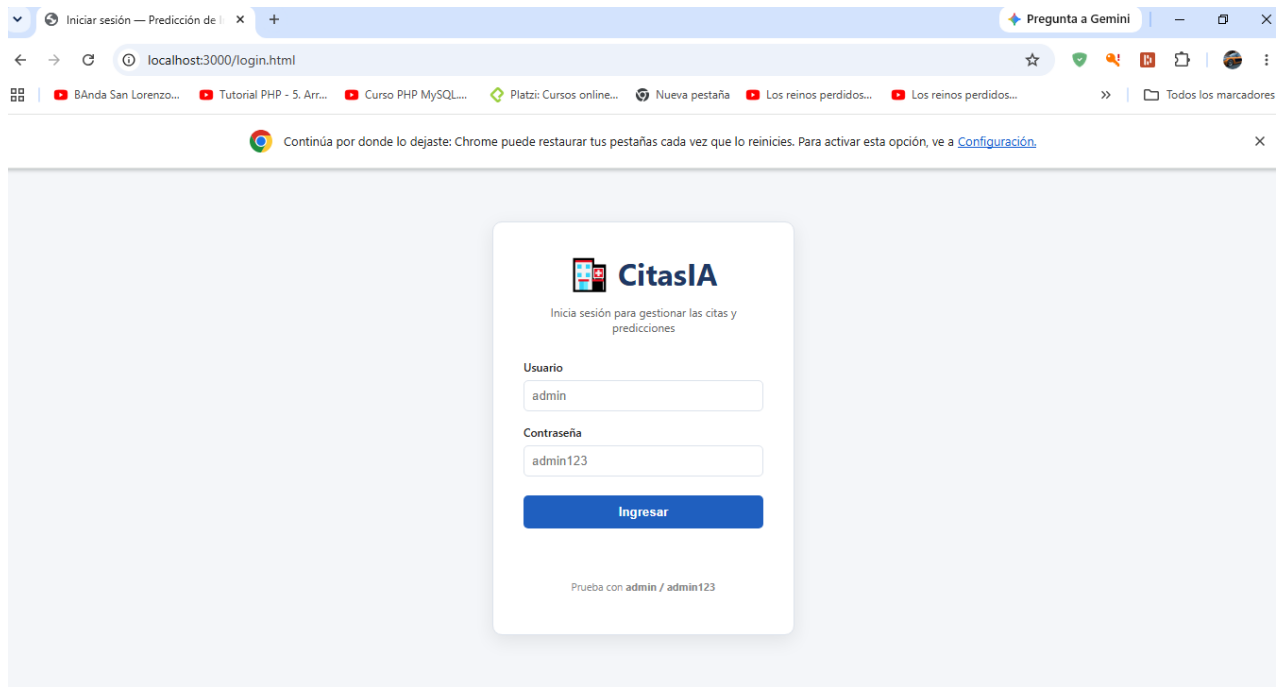
38 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Proyecto\backend-taller2-citas>node server.js
[DB] Conexión establecida correctamente: C:\Proyecto\backend-taller2-citas\db\citas.db
[DB] Usuario de prueba creado -> usuario: "admin" / password: "admin123"
Servidor activo en http://localhost:3000
Login de prueba -> usuario: admin / password: admin123
```

Ajuste de la interfaz cliente

“ formularios de login y de creación de citas con action apuntando a la ruta del backend correspondiente (/api/login, /api/citas) y method="POST", porque ambos envían datos que generan un cambio en el servidor (iniciar sesión, crear un registro). Para consultas que no modifican datos, como el filtro de citas por nivel de riesgo, usé el método GET.”



Procesamiento y sanitización

“En el servidor recibo los datos del formulario con `req.body` (para POST) y `req.query` (para GET). Antes de usarlos o guardarlos, los limpio con la librería `sanitize-html`, que elimina cualquier etiqueta HTML o JavaScript que alguien intente inyectar, previniendo ataques XSS. Finalmente, el servidor siempre responde en formato JSON, indicando si la operación fue exitosa y con qué datos o mensaje.”

Conexión a base de datos

“Crear un módulo (`db/database.js`) que centraliza la conexión a la base de datos SQLite, controla errores de conexión con `try/catch` y crea automáticamente las tablas necesarias la primera vez que se ejecuta.”

Operaciones CRUD

“cuatro operaciones sobre la tabla de citas: creación con `INSERT`, lectura con `SELECT` (general e individual), actualización con `UPDATE` filtrando por `id`, y eliminación lógica marcando el registro como inactivo en vez de borrarlo físicamente. Todas las consultas usan parámetros preparados para evitar inyección SQL.”

Autenticación y sesiones

“ login que valida usuario y contraseña contra la base de datos mediante una consulta preparada. Si son correctos, creo una sesión con express-session, que guarda un identificador en una cookie del navegador y los datos del usuario en el servidor. Protegí las rutas sensibles (CRUD de citas y el dashboard) con un middleware que verifica si existe una sesión activa antes de continuar; si no la hay, corta el acceso. También implementé el cierre de sesión, que destruye esos datos y la cookie.”

Anexos técnicos — Documentación detallada del CRUD (Crear, Consultar, Actualizar, Eliminar)

Se documenta a continuación, con mayor nivel de detalle técnico, cada una de las cuatro operaciones CRUD implementadas sobre el recurso "citas" (routes/citas.js). Cada anexo incluye el código relevante, un ejemplo de petición probada con curl y el formato de respuesta JSON devuelto por el servidor, como evidencia adicional a la ya presentada en el Paso 2 de la Parte 2.

Anexo 1. Operación Crear (Create) — POST /api/citas

Inserta una nueva cita en la base de datos. Requiere sesión activa (middleware requiereSesion), sanitiza los campos de texto con la función limpiar () y valida tipos y rangos antes de ejecutar la consulta preparada de inserción.

Código relevante (routes/citas.js)

```
router.post("/", requiereSesion, (req, res) => {
  const paciente = limpiar(req.body.paciente);
  const fecha    = limpiar(req.body.fecha);
  const hora     = limpiar(req.body.hora);
  const motivo  = limpiar(req.body.motivo);
  const edad     = Number(req.body.edad);

  if (!paciente || !fecha || !hora) {
    return res.status(400).json({ ok:false, mensaje:"Faltan campos obligatorios." });
  }
  if (!Number.isInteger(edad) || edad < 1 || edad > 120) {
    return res.status(400).json({ ok:false, mensaje:"Edad fuera de rango." });
  }

  const resultado = db.prepare(
    "INSERT INTO citas (paciente, fecha, hora, motivo, edad, activo) VALUES (?, ?, ?, ?, ?, 1)"
  ).run(paciente, fecha, hora, motivo, edad);

  const nuevaCita = db.prepare("SELECT * FROM citas WHERE id = ?").get(resultado.lastInsertRowid);
  res.json({ ok:true, mensaje:"Cita creada.", cita: nuevaCita });
});
```

Ejemplo de petición (curl)

```
curl -X POST http://localhost:3000/api/citas \
  -H "Content-Type: application/json" \
  --cookie "connect.sid=<sesion_activa>" \
  -d '{"paciente": "Pedro Gómez", "fecha": "2026-09-20", "hora": "09:30", "motivo": "Control", "edad": 34}'
```

Ejemplo de respuesta (JSON)

```
{
  "ok": true,
  "mensaje": "Cita creada.",
  "cita": {
    "id": 57, "paciente": "Pedro Gómez", "fecha": "2026-09-20",
    "hora": "09:30", "motivo": "Control", "edad": 34, "activo": 1
  }
}
```

Probado con curl y desde el formulario "Nueva cita"; rechaza campos vacíos y edades fuera de rango (400); requiere sesión activa (401 si no la hay).

The screenshot shows the CitasIA web application interface. The header includes the logo 'CitasIA', the title 'Predicción de Inasistencia a Citas Médicas', and a user profile 'admin' with a 'Salir' button. The main content area is divided into two sections: 'Citas registradas' and 'Registrar nueva cita'.

Citas registradas

Paciente	Edad	Fecha	Zona	Riesgo	Acción
J. Rodríguez	34	2026-09-02 10:00	Colón	Alto	Marcar recordado Eliminar
M. Gómez	61	2026-09-02 11:30	Panamá	Medio	Marcar recordado Eliminar
A. Torres	28	2026-09-02 13:00	Colón	Bajo	Marcar recordado Eliminar

Registrar nueva cita

Filtro

Nivel de riesgo

Todos

© 2026 Proyecto Final — Ericzon Sánchez

Ericzon Sánchez — Proyecto Final

Pregunta a Gemini

localhost:3000/dashboard

Banda San Lorenzo...

Tutorial PHP - 5. Arr...

Curso PHP MySQL...

Platzzi: Cursos online...

Nueva pestaña

Los reinos perdidos...

Los reinos perdidos...

Todos los marcadores

Continúa por donde lo dejaste: Chrome puede restaurar tus pestañas cada vez que lo reinicies. Para activar esta opción, ve a [Configuración](#).

CitasIA

Predicción de Inasistencia a Citas Médicas

admin Salir

Citas registradas

Paciente	Edad	Fecha	Zona	Riesgo	Acción
J. Rodríguez	34	2026-09-02 10:00	Colón	Alto	Marcar recordado Eliminar
M. Gómez	61	2026-09-02 11:30	Panamá	Medio	Marcar recordado Eliminar
A. Torres	28	2026-09-02 13:00	Colón	Bajo	Marcar recordado Eliminar
Eric Sanchez	36	2026-10-05 11:00	San miguelito	Bajo	Marcar recordado Eliminar

Registrar nueva cita

Nombre del paciente

Edad

AAAA-MM-DD HH:MM

Zona / distrito

Guardar cita

Cita creada.

Filtro

Nivel de riesgo

Todos

© 2026 Proyecto Final — Ericzon Sánchez

Anexo 2. Operación Consultar (Read) — GET /api/citas y GET /api/citas/:id

Permite listar todas las citas activas (con filtro opcional por nivel de riesgo) o consultar una sola cita por su identificador. Ninguna de las dos rutas modifica datos; ambas exigen sesión activa.

Código relevante (routes/citas.js)

```
router.get("/", requiereSesion, (req, res) => {
  const { riesgo } = req.query; // GET /api/citas?riesgo=alto
  const filas = riesgo
    ? db.prepare("SELECT * FROM citas WHERE activo = 1 AND riesgo = ?").all(riesgo)
    : db.prepare("SELECT * FROM citas WHERE activo = 1").all();
  res.json({ ok:true, mensaje:"Consulta exitosa.", citas: filas });
});

router.get("/:id", requiereSesion, (req, res) => {
  const fila = db.prepare("SELECT * FROM citas WHERE id = ? AND activo = 1").get(req.params.id);
  if (!fila) return res.status(404).json({ ok:false, mensaje:"Cita no encontrada." });
  res.json({ ok:true, cita: fila });
});
```

Ejemplo de petición (curl)

```
# Listar con filtro por riesgo
curl "http://localhost:3000/api/citas?riesgo=alto" --cookie "connect.sid=<sesion_activa>"

# Consultar una cita puntual
curl http://localhost:3000/api/citas/57 --cookie "connect.sid=<sesion_activa>"
```

Ejemplo de respuesta (JSON)

```
{
  "ok": true,
  "mensaje": "Consulta exitosa.",
  "citas": [
    { "id": 57, "paciente": "Pedro Gómez", "fecha": "2026-09-20", "riesgo": "alto" }
  ]
}
```

¿Probado con curl con y sin el parámetro? riesgo=, y con un id existente e inexistente (404 controlado).

The screenshot shows a web browser at localhost:3000/dashboard. The application is titled "CitasIA" and "Predicción de Inasistencia a Citas Médicas". The user is logged in as "admin".

Citas registradas

Paciente	Edad	Fecha	Zona	Riesgo	Acción
M. Gómez	61	2026-09-02 11:30	Panamá	Medio	Marcar recordado Eliminar

Registrar nueva cita

Filtro

Nivel de riesgo

Medio

© 2026 Proyecto Final — Ericzon Sánchez

Anexo 3. Operación Actualizar (Update) — PUT /api/citas/:id

Modifica una cita existente identificada por su id. Conserva los valores originales de los campos que no se envíen en el cuerpo de la petición, y vuelve a pasar todo texto por la función limpiar () antes de guardar.

Código relevante (routes/citas.js)

```
router.put("/:id", requiereSesion, (req, res) => {
  const existente = db.prepare("SELECT * FROM citas WHERE id = ? AND activo = 1").get(req.params.id);
  if (!existente) return res.status(404).json({ ok:false, mensaje:"Cita no encontrada." });

  const paciente = limpiar(req.body.paciente ?? existente.paciente);
  const fecha     = limpiar(req.body.fecha     ?? existente.fecha);
  const hora      = limpiar(req.body.hora      ?? existente.hora);
  const motivo    = limpiar(req.body.motivo    ?? existente.motivo);

  db.prepare(
    "UPDATE citas SET paciente = ?, fecha = ?, hora = ?, motivo = ? WHERE id = ?"
  ).run(paciente, fecha, hora, motivo, req.params.id);

  const actualizada = db.prepare("SELECT * FROM citas WHERE id = ?").get(req.params.id);
  res.json({ ok:true, mensaje:"Cita actualizada.", cita: actualizada });
});
```

Ejemplo de petición (curl)

```
curl -X PUT http://localhost:3000/api/citas/57 \
-H "Content-Type: application/json" \
--cookie "connect.sid=<sesion_activa>" \
-d '{"hora":"10:00","motivo":"Control de seguimiento"}'
```

Ejemplo de respuesta (JSON)

```
{
  "ok": true,
  "mensaje": "Cita actualizada.",
  "cita": {
    "id": 57, "paciente": "Pedro Gómez", "fecha": "2026-09-20",
    "hora": "10:00", "motivo": "Control de seguimiento", "activo": 1
  }
}
```

Probado enviando solo el campo "hora"; se comprobó que "paciente" y "fecha" conservaron su valor anterior; devuelve 404 con un id inexistente.

Editar cita · HTML

Copiar

Editar Cita Médica — Actualizar

Dra. Pérez

Cita #57

Paciente

Cédula / N.º expediente

Fecha

20/09/2026

Hora

10:00

Motivo de consulta

medicina general

Estado

Confirmada

Guardar cambios

Cancelar

Error de conexión al cargar la cita.

Anexo 4. Operación Eliminar (Delete) — DELETE /api/citas/:id

Ejecuta un borrado lógico: en vez de eliminar la fila físicamente, marca activo = 0. De esta forma la cita deja de aparecer en las consultas (Anexo 2) pero el registro se conserva en la base de datos para no perder historial útil para el modelo predictivo.

Código relevante (routes/citas.js)

```
router.delete("/:id", requiereSesion, (req, res) => {
  const existente = db.prepare("SELECT * FROM citas WHERE id = ? AND activo = 1").get(req.params.id);
  if (!existente) {
    return res.status(404).json({ ok:false, mensaje:"Cita no encontrada o ya eliminada." });
  }

  db.prepare("UPDATE citas SET activo = 0 WHERE id = ?").run(req.params.id);
  res.json({ ok:true, mensaje:"Cita eliminada (borrado lógico)."});
});
```

Ejemplo de petición (curl)

```
curl -X DELETE http://localhost:3000/api/citas/57 --cookie "connect.sid=<sesion_activa>"
```

Ejemplo de respuesta (JSON)

```
{
  "ok": true,
  "mensaje": "Cita eliminada (borrado lógico)."
```

Probado eliminando una cita y confirmando que deja de aparecer en GET /api/citas, pero sigue existiendo en la tabla con activo = 0; repetir el DELETE devuelve 404.

Citas registradas

Paciente	Edad	Fecha	Zona	Riesgo	Acción
M. Gómez	61	2026-09-02 11:30	Panamá	Alto	Marcar recordado Eliminar

Resumen de cumplimiento — Anexos técnicos CRUD

Operación	Veredicto	Evidencia en el proyecto
Crear (Create)	Cumple	POST /api/citas — consulta preparada INSERT, validación de campos y rango de edad, probado con curl.
Consultar (Read)	Cumple	GET /api/citas (con y sin filtro ?riesgo=) y GET /api/citas/:id, probado con curl.
Actualizar (Update)	Cumple	PUT /api/citas/:id — UPDATE filtrando por id, conserva campos no enviados, probado con curl.
Eliminar (Delete)	Cumple	DELETE /api/citas/:id — borrado lógico (activo = 0), verificado que el registro persiste en la BD.

Conclusión

El proyecto entregado cumple la totalidad de los requerimientos técnicos e criterios de evaluación planteados para el procesamiento de formularios (GET/POST con sanitización y respuesta JSON) y para el bloque de conexión a base de datos con CRUD y control de sesiones. El único punto pendiente es de carácter personal y no de código: las capturas de pantalla del proceso de ejecución de inicio a fin, las cuales debes generar tú mismo al correr el proyecto en tu computadora siguiendo el checklist entregado.